REQUIREMENTS SPECIFICATION

Work Schedule Generator

Constraint-based automated scheduling for quick-service restaurant store management

Version	Date	Audience	Status
0.2 — Draft	June 13, 2026	Developers / Contractors	Scoped, pre-build

1 Purpose & Overview

The Schedule Generator automatically produces weekly staff schedules that satisfy store coverage requirements given the available people, their employment status, availability, and preferences. The primary objective is to dramatically reduce the time a store manager spends building schedules by hand, replacing a lengthy manual process with constraint-based optimization plus a manager-friendly editing interface.

The tool is an **upstream pre-builder**: it generates and refines an optimized schedule which is then **exported into Taco Bell Tracks** (the existing labor-management system) for downstream cost calculation, statistics, and payroll. Tracks remains the system of record for labor cost, SPH/TPH, variance, and coverage analytics; this tool's responsibility ends at producing a valid, optimized, import-ready schedule. It therefore deliberately excludes those downstream analytics from its own scope.

The system serves two equally weighted core workflows: generating an optimal schedule from a blank slate, and updating a previously generated schedule by applying a queue of personnel changes with minimal disruption to existing assignments. In both cases the solver does not fail outright when it cannot satisfy every requirement; instead it surfaces unmet items as a **gap report** for the manager to resolve manually, keeping the human in control.

> **Design philosophy — soft constraints over hard failures.** Wherever a requirement is a preference rather than a legal or operational necessity, the solver treats it as a weighted objective. It produces the best feasible schedule it can and reports what it could not satisfy, rather than returning no schedule at all.

2 Operating Parameters

Store hours	5:00 AM – 12:30 AM (next day), daily
Scheduling granularity	15-minute slots (82 slots per day)
Daily labor target	Minimum 70 hrs · soft cap 75 hrs · hard cap 80 hrs *(total store labor per day, summed across all staff)*
Manager presence	At least 1 manager on site at all times the store is open

3 Coverage & Shift Constraints

Coverage requirements define how many staff must (or should) be scheduled in any given 15-minute slot. Each is classified as a **HARD** constraint (must be satisfied or reported as a blocking gap) or a **SOFT** constraint (optimized toward; shortfalls reported but non-blocking).

3.1 Rush coverage SOFT

Target staffing of **5 people** during rush periods, every day:

- Lunch rush: 11:00 AM – 1:00 PM
- Dinner rush: 6:00 PM – 8:00 PM

5 is a preferred target, not a hard floor. The solver optimizes toward 5 and reports any rush slot staffed below it.

3.2 Baseline coverage SOFT / HARD FLOOR

Outside of rush and late-night windows, staff **at least 3** (hard floor), but **prefer 4**. The solver attempts to reach 4 and reports any slot that falls below 4.

3.3 Late-night reduced coverage HARD CAP

After a per-day cutoff time, no more than **2 staff** are scheduled (the closing crew). Cutoffs vary by day:

Day	Cutoff — max 2 staff after
Monday	10:00 PM
Tuesday	11:00 PM
Wednesday	10:30 PM
Thursday	11:00 PM
Friday – Sunday	11:30 PM

> **Import format dependency.** The late-night cap is treated as a hard cap of 2 staff after each day's cutoff. The export that feeds Tracks must be an exact, import-ready match to the format Tracks expects (see §7.4); that format must be reverse-engineered from a real Tracks export before the export module is finalized.

4 Shift Rules

Rule	Detail
Regular shift length	4 – 8.5 hours
Manager (GM) shift length	Up to 10.5 hours
Unpaid lunch	30-minute unpaid break auto-inserted for any shift over 5 hours

Rush staffing method	Achieved through naturally overlapping shifts, not hard start/stop boundaries
Minor — school week (school nights)	Cannot work more than 4 hours, and cannot work past 10:00 PM

5 People & Personnel Inputs

Each employee carries the attributes the solver needs to place them correctly:

- **General availability** — recurring weekly windows the person can work.
- **Status** — employment type and standing (full-time / part-time, manager, minor, etc.), feeding both hard rules and priority weighting.
- **Preferences** — soft inputs the solver honors where possible.
- **Hard-set assignments** — the manager can lock a specific person into a specific shift/slot, which the solver treats as a fixed constant rather than something to optimize. Used for recurring fixed schedules (e.g. the GM) with per-day, one-week override support that does not alter the underlying template.

5.1 Priority weighting

When distributing hours, the solver weights employees by a combination of employment type (FT vs. PT), seniority, certifications, and performance, so that hours flow to higher-priority staff first within the constraints.

5.2 Positions — out of solver scope

In Tracks, every shift carries a position label (e.g. MIC, Food Champion, Service Champion, Prep, RGM Tasks). For this tool's solver, **only manager presence is a scheduling constraint** (§2). The solver does not assign or optimize the other positions; position labelling, if needed, is handled downstream in Tracks or manually by the manager.

6 Personnel Changes (Queued)

The system can load a previously generated schedule and queue a set of changes against it, then re-solve incrementally around the existing assignments. Supported change types:

- **Day-off requests** — whole-day or partial-day.
- **Termination / suspension** — remove a person from scheduling going forward.
- **Leave of absence.**
- **Change of availability** — update a person's recurring windows.

> **Incremental re-solve goal:** apply queued changes while disturbing the rest of the existing schedule as little as possible, rather than regenerating from scratch.

7 Output & Editing Interface

The tool reproduces the three schedule views used in Tracks, plus the manual slider editor from the original scope. All views render the same underlying schedule.

7.1 Grid editor

A weekly employee × day matrix with selectable options — the primary view of the generated schedule. Each cell shows the shift's start/end time for that employee on that day. This is the main surface for reviewing and selecting assignments.

7.2 Timeline (Gantt) view

A daily horizontal timeline spanning store hours (5:00 AM – 12:30 AM), with one row per employee and shift blocks drawn across the time axis. Used to visually assess coverage density across the day, including rush windows. Read-oriented for spotting over/under-coverage at a glance.

7.3 Printable report

A weekly tabular report (employee × day) with start, end, and daily-total columns plus a weekly per-employee total — suitable for printing or PDF export, matching the "Weekly Labor Schedule With Daily Total" layout staff are already familiar with.

7.4 Tracks export HARD

An **exact, import-ready export** that loads directly into Taco Bell Tracks with no manual reformatting. The export must match Tracks' expected import schema field-for-field. *The precise format is to be reverse-engineered from a real Tracks export file before this module is finalized.*

7.5 Slider editor

Manual adjustment of individual shifts with live validation as the manager drags start/end times, surfacing any newly created constraint violations immediately.

7.6 Gap report

Unmet soft constraints surfaced to the manager for manual resolution rather than presented as hard failures.

8 Core Functions

ID	Function
F-1	**Generate optimal schedule from blank.** Produce a full weekly schedule satisfying all hard constraints and optimizing soft ones, then identify and report remaining errors / unmet requirements.
F-2	**Update previous schedule from queued changes.** Load an existing schedule, apply the change queue via incremental re-solve, then identify and report remaining errors / unmet requirements.

F-1 and F-2 are equal priority for v1.

9 Functional Requirements Summary

ID	Requirement	Priority
R-01	Generate a feasible weekly schedule from no prior assignments.	Must
R-02	Enforce ≥1 manager present during all open hours.	Must
R-03	Enforce per-day late-night cap of max 2 staff after each cutoff.	Must
R-04	Enforce baseline hard floor of ≥3 staff outside rush/late-night.	Must
R-05	Optimize toward 4 baseline / 5 rush; report shortfalls.	Should
R-06	Enforce shift-length bounds, GM exception, and auto-lunch insertion.	Must
R-07	Enforce minor school-night limits (≤4 hrs, not past 10 PM).	Must
R-08	Keep daily labor within 70 (min) / 75 (soft) / 80 (hard) hours.	Should
R-09	Honor availability and hard-set assignments as fixed constants.	Must
R-10	Weight hour distribution by type, seniority, certs, performance.	Should
R-11	Load a previous schedule and queue personnel changes.	Must
R-12	Incrementally re-solve with minimal disruption to existing shifts.	Must
R-13	Provide grid editor with selectable options.	Must
R-14	Provide slider editor with live validation.	Should
R-15	Surface a gap report of unmet soft constraints.	Must
R-16	Provide daily timeline (Gantt) view across store hours.	Should
R-17	Provide printable weekly report with daily and weekly totals.	Should
R-18	Produce an exact, import-ready export for Taco Bell Tracks.	Must

10 Technical Architecture

Frontend	Next.js, deployed on Vercel
Solver	Python + OR-Tools CP-SAT, Dockerized, deployed on AWS Lambda or Fargate
Job queue	AWS SQS for async job handling
Database	AWS RDS PostgreSQL with RDS Proxy
ORM (app)	Prisma (TypeScript) — single source of truth for schema & migrations

ORM (solver)	SQLAlchemy (Python solver service only)
Local dev	Docker Compose for production parity

PostgreSQL chosen for relational fit with the scheduling domain, ACID guarantees for atomic schedule publishing, JSONB for gap-report storage, and clean Docker-to-RDS parity.

10.1 Build order

Foundation-first: the core engine is proven solid before any editing UI is layered on.

#	Stage
1	Data model
2	Validation engine
3	Solver prototype
4	Async job wiring
5	Grid editor
6	Slider editor
7	Incremental re-solve mode